ProtecPhone

Projet NoSQL / Architecture Microservices

EFREI Paris • Master 1

Professeur : M. Charroux

Remise prévue : 15 novembre 2026

Membres du groupe :

SERRAKH Touria

Ilyesse El Adaoui

BALLUAIS Matteo

DAOUD Sheyma

Application Java 21 / Spring Boot • Redis • MongoDB

Ce rapport présente l'implémentation vérifiée du dépôt ProtecPhone : trois services, des agrégats métier imbriqués et deux répliques mises à jour par événements. La finalisation conserve le travail Redis du groupe et ajoute un stockage documentaire MongoDB avec une reprise explicite des écritures asynchrones.

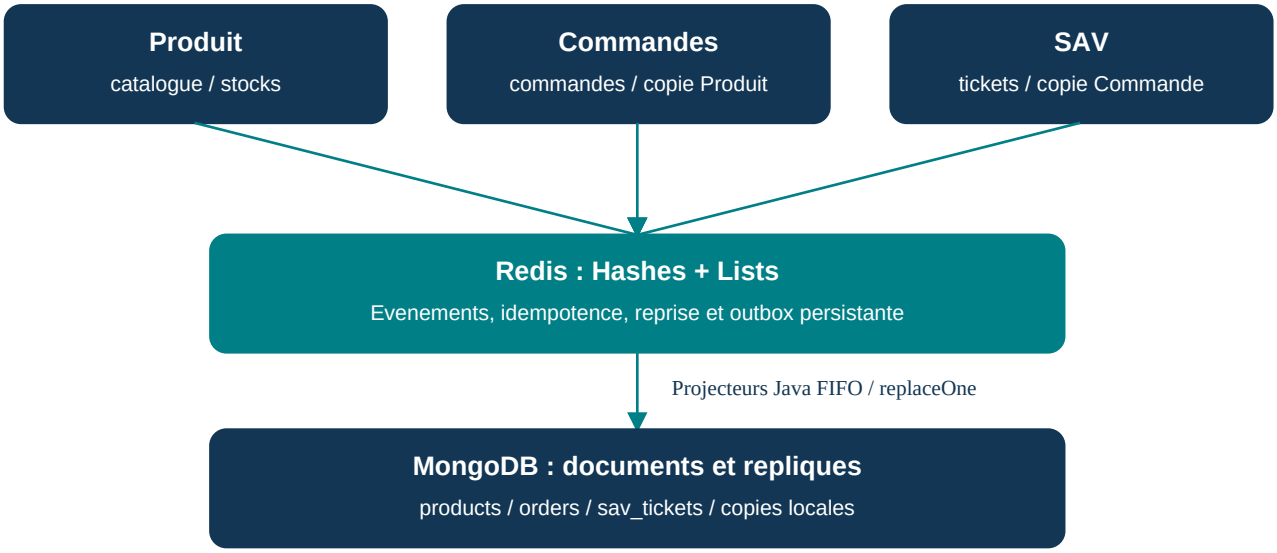
Dépôt : https://github.com/mballuais/ProtecPhone

Version documentaire : 29 octobre 2026. Les traces de validation sont conservées dans docs/validation/ ; les exemples correspondent au scénario reproductible demo.

1. Application et architecture

ProtecPhone modélise la vente de coques de smartphones et le suivi après-vente. Le périmètre implémenté comprend les API de catalogue, les commandes avec calcul du total, les changements de statut, les réservations/restitutions de stock et les tickets SAV. Les clients de démonstration sont fictifs. Aucun frontend, paiement ou moteur de recommandation n'est nécessaire au scénario présenté.

Les trois services sont des conteneurs indépendants construits depuis le même artefact Maven. La variable SERVICE_ROLE sélectionne les routes et le consommateur. Les données sont possédées logiquement par un seul service ; les autres en conservent seulement les copies nécessaires à leurs traitements.



Service	Responsabilité	Données possédées / copiées
Produit	Catalogue, prix et stock par couleur	Produit ; registre d'allocation
Commandes	Validation des lignes, total et statut	Commande ; copie des Produits
SAV	Tickets et échanges de messages	Ticket ; copie des Commandes

Redis porte les états opérationnels et les files d'événements. MongoDB conserve les documents métier et les copies locales, réellement écrits et lus par les services Java. Les projecteurs Java consomment une outbox persistante dans Redis. La configuration utilise un serveur de chaque technologie et un volume persistant par base. La séparation des données est logique, sans contrôle d'accès par service au niveau des bases dans cette démonstration locale.

2. Agrégats métier et répliques

Les fichiers JSON fournis restent inchangés. L'agrégat Produit regroupe les modèles compatibles et les variantes : le prix et le stock dépendent de la couleur. Extrait du premier produit du scénario, avant commande :

```
{
  "produitId": "coque-001",
  "nom": "Coque transparente renforcée",
  "compatibilite": ["iPhone 15"],
  "variantes": [{"couleur": "noir", "prix": 14.99, "stock": 80}]
}
```

L'agrégat Commande regroupe identité/date, client, lignes, adresse, statut et total. Le client reste imbriqué, sans collection Client distincte. CMD-42 contient CLI-7, Marie Dupont, et deux coques-001 noires pour iPhone 15 à 14.99 euros, soit 29.98 euros. Chaque ligne conserve le nom, le modèle, la couleur, la quantité et le prix unitaire au moment de l'achat. Java vérifie ces informations sur la copie locale du Produit.

L'agrégat SAV contient ticketId, commandeId, dateCreation, motif, statut et messages. SAV-9 concerne une coque fissurée de CMD-42 ; SAV-10, une mauvaise couleur de CMD-43. Les messages contiennent auteur, texte et date ISO-8601.

Réplique	Contenu conservé	Alimentation
Produit dans Commandes	Produit complet et variantes	Événements Produit
Commande dans SAV	Client, lignes, date, total, statut ; sans adresse	Événements Commande

La copie Produit permet à Commandes de valider une commande sans appel HTTP direct à Produit. La copie Commande permet au SAV de vérifier l'existence d'une commande sans dépendre de sa disponibilité immédiate. Il s'agit de cohérence éventuelle : la création HTTP et l'arrivée de la réplique sont deux instants distincts.

Les schémas de référence sont Aggregat-Produit.json, Aggregat-Commande.json, Aggregat-SAV.json et les deux fichiers Replica-*.json. Les enveloppes Commande-*.json restent acceptées par POST /commands ; le scénario réutilise leurs identifiants.

3. Redis : état opérationnel et événements

Redis est adapté aux lectures par identifiant et au traitement ordonné des événements. Les Hashes conservent un champ par propriété métier de premier niveau. Les objets client et les tableaux lignes, variantes et messages sont des chaînes JSON dans leurs champs respectifs. Ils sont désérialisés par Java sans modifier le modèle métier.

Structure	Exemple de suffixe	Usage
Hash	product:coque-001 ; order:CMD-42 ; sav:SAV-9	Agrégats opérationnels
Hash	replica:product:coque-001 ; replica:order:CMD-42	Copies locales
List	events ; queue:orders ; processing:orders	Journal, attente et reprise
Hash	processed:orders ; metrics:orders	Déduplication et compteur
List	mongo:queue:orders ; mongo:processing:orders	Outbox documentaire

Tous ces suffixes sont précédés de protecphone:demo: dans Compose. Les écritures métier, les événements et l'ajout à l'outbox sont regroupés dans le même script Lua. Le journal events reste consultable après consommation des files.

```
docker compose exec -T redis redis-cli HGET
  protecphone:demo:order:CMD-42 statut
Résultat après demo : EXPEDIEE

docker compose exec -T redis redis-cli HEXISTS
  protecphone:demo:order:CMD-42 commandeId
Résultat : 1
```

Les retours à la ligne ci-dessus servent à la mise en page ; les commandes complètes sur une ligne figurent dans docs/REQUETES-NOSQL.md, avec HGETALL, LRANGE et LLEN. Après la démonstration, LLEN sur events vaut 14 et les files sont vides.

La persistance AOF est activée avec appendfsync always sur le volume redis-data. LMOVE déplace atomiquement un message de queue vers processing. Le consommateur reprend processing avant d'en réclamer un autre. Le traitement métier, le marquage processed et l'acquittement partagent le commit Lua ; un même eventId n'a pas de second effet. Le registre d'allocation protège aussi les restitutions de stock.

4. MongoDB : documents métier interrogeables

MongoDB conserve naturellement les objets et tableaux imbriqués des agrégats. Une commande et ses lignes se lisent dans un document, sans jointure. Le SAV interroge sa copie locale de commande. Ce choix complète les accès opérationnels et les files Redis avec des filtres documentaires sur le client, les variantes ou le statut.

Collection	Propriétaire	Contenu après demo
products	Produit	5 produits
orders	Commandes	3 commandes
sav_tickets	SAV	2 tickets
product_replicas	Commandes	5 copies Produit
order_replicas	SAV	3 copies Commande

La base est protecphone. Les collections stock_allocations et sav_status_replicas conservent aussi les registres auxiliaires utilisés par les services. Un document possède les champs métier à la racine et deux métadonnées : _id, égal à la clé Redis complète, et _namespace, égal à protecphone:demo. L'index unique _id évite les doublons lors d'un upsert. Exemple d'identifiant : protecphone:demo:order:CMD-42.

```
db.orders.findOne({
  _namespace: 'protecphone:demo', commandeId: 'CMD-42'
})
// CMD-42, CLI-7, total 29.98, statut EXPEDIEE

db.orders.find({
  _namespace: 'protecphone:demo', 'client.clientId': 'CLI-7'
}).toArray()
// une commande : CMD-42
```

Ces requêtes sont exécutables dans mongosh du conteneur. Les commandes Docker complètes et leurs résultats réels sont dans docs/REQUETES-NOSQL.md et docs/validation/requetes.log. L'API GET /documents/orders/CMD-42 consulte MongoDB depuis le service Commandes ; les accès à une collection d'un autre rôle sont refusés.

Le volume mongo-data et les écritures journalisées protègent les redémarrages testés. Les dates restent des chaînes ISO-8601. Le calcul Java utilise BigDecimal ; les montants projetés sont des nombres BSON double, suffisants pour cette démonstration. Une application financière devrait fixer un stockage Decimal128 ou en centimes.

5. Synchronisation et reprise sur panne

Une double écriture HTTP naïve Redis puis MongoDB laisserait un état incomplet si la seconde base échouait. L'implémentation choisit Redis comme point de validation opérationnel et inscrit les snapshots à projeter dans une outbox au même commit. MongoDB est mis à jour par un worker Java indépendant pour chaque rôle.

1. Produit crée coque-001 et publie ProductCreated. Commandes traite l'événement, crée son Hash replica:product:coque-001 et enregistre sa projection MongoDB. 2. Commandes crée CMD-42 avec deux unités. OrderCreated est envoyé à Produit et SAV. Produit réserve le stock (80 devient 78), puis publie StockChanged ; Commandes met à jour sa copie. SAV conserve la commande sans adresse de livraison. 3. CMD-42 passe à EXPEDIEE. OrderStatusChanged modifie le statut du Hash de la copie SAV et ajoute son document actualisé à l'outbox ; MongoDB converge vers EXPEDIEE.

Le projecteur prend le plus ancien lot avec LMOVE, puis fait des replaceOne par _id avec upsert=true. Il acquitte seulement après toutes les écritures du lot. S'il s'interrompt, le même lot est repris : remplacer un document par le même snapshot est idempotent. Aucun calcul de stock n'est répété dans MongoDB. Une seule instance par rôle maintient l'ordre FIFO, y compris pendant la reprise.

Incident	Comportement
MongoDB arrêté	Redis accepte l'écriture et garde l'outbox ; reprise automatique
Arrêt après écriture Mongo, avant acquittement	Même remplacement rejoué sans doublon
Service indisponible	Événements conservés dans sa queue ; consommation au retour
Message invalide	Conservation dans dead avec erreur ; aucune disparition silencieuse

Le système garantit une convergence après traitement, sans transaction distribuée. Un lot multi-document peut être partiellement visible pendant sa projection. La réponse HTTP confirme Redis et l'outbox, pas l'achèvement MongoDB. Les tests attendent le vidage des files puis comparent les documents des deux bases.

Limites : pas de haute disponibilité, pas de saga de compensation des surventes concurrentes, pas de rotation automatique des journaux, pas de version de snapshot pour un ancien événement artificiellement réémis sous un nouvel eventId. La perte des volumes n'est pas couverte. Les clés réservées doivent conserver les bons types : une erreur Lua d'exécution ne bénéficie pas d'un rollback Redis.

6. Validation, démonstration et remise

La validation utilise de vrais conteneurs Redis, MongoDB et trois services Spring Boot, sans bases simulées. Le journal intégral et le code de sortie sont sauvegardés dans docs/validation/integration.log et result.json. Le document docs/TESTS.md précise les résultats, la configuration et les limites de couverture.

Validation réelle : 18 tests d'intégration réussis, puis tous les scénarios de panne et de reprise terminés avec le code 0. Fin de l'exécution : 2026-10-29T09:03:36.152205+00:00.

Le scénario crée cinq produits, trois commandes et deux tickets, vérifie les deux copies, puis l'expédition de CMD-42. Il compare les cinq collections métier avec les Hashes Redis, exécute les requêtes mongosh et vérifie les comptes 5/3/2/5/3. Les tests couvrent les événements, prix, stocks, annulations, retours, entrées invalides, doublons, droits de lecture par rôle et rejeu de projection. Les tests de panne exercent l'arrêt d'un consommateur, un message en cours, le redémarrage de Redis, celui de MongoDB, une panne MongoDB avec redémarrage du propriétaire, et un SIGKILL.

```
docker compose up --build -d
python scripts/run.py demo
python scripts/validate.py
```

Python orchestre les commandes ; Java/Maven sont fournis par Docker. Les ports hôte par défaut sont 8081/8082/8083 et 6379, configurables par variables. MongoDB reste sur le réseau Docker. La validation de cette mission utilise les ports 28081/28082/28083 et 6380 pour préserver les autres projets de l'ordinateur.

Le rechargement arrête les écrivains et ne supprime que le namespace réservé à la démonstration, dans les deux bases. Il est relançable sans doublons. Les tests modifient volontairement le jeu ; relancer demo avant la soutenance.

Les cinq exigences techniques du cours sont satisfaites par les trois services, deux technologies NoSQL, trois agrégats propriétaires, deux copies locales et leur mise à jour événementielle. L'application illustre la cohérence éventuelle et une reprise explicite entre stockage opérationnel et documentaire. La validation locale ne constitue pas une certification de charge ni de sécurité de production.

Livrables : ce PDF et sa source éditable, le dépôt Git, les preuves de validation et une archive de remise avec instructions. L'étudiant doit vérifier la version publiée et téléverser les fichiers sur Moodle ; aucun dépôt Moodle n'est annoncé sans confirmation réelle.